PROYECTO DE BASE DE DATOS

Sistema de gestión para una heladería

Modelado, normalización e implementación en MySQL Workbench

ASIGNATURA	Base de Datos 2
FECHA DE ENTREGA	Jueves, 15 de octubre de 2026
HERRAMIENTA PRINCIPAL	MySQL Workbench
INTEGRANTE	Johan Sebastian Almeida Bobailla
DOCENTE	Giovanny Rubiano

Contenido

1. Introducción . . . 3

Alcance del sistema . . . 3

Propósito principal . . . 3

2. Planteamiento del problema . . . 4

Situación actual . . . 4

Problemas identificados . . . 4

Justificación de la solución propuesta . . . 4

Pregunta guía . . . 4

3. Macro con todos los módulos . . . 5

Interacción entre módulos . . . 5

4. Objetivos . . . 6

Objetivo general . . . 6

Objetivos específicos . . . 6

5. Diccionario de datos . . . 7

Convenciones y supuestos . . . 7

6. Modelo Entidad-Relación (MER) . . . 11

Relaciones y cardinalidades . . . 11

7. Modelo Relacional inicial (MR) . . . 13

8. Proceso de normalización . . . 14

8.1 Punto de partida: ficha de pedido sin normalizar (UNF) . . . 14

8.2 Primera forma normal (1FN) . . . 14

8.3 Segunda forma normal (2FN) . . . 14

8.4 Tercera forma normal (3FN) . . . 15

8.5 Aplicación al MR inicial . . . 15

8.6 Verificación final del modelo . . . 16

9. Modelo Relacional corregido (tras la normalización) . . . 17

10. Implementación en MySQL Workbench . . . 18

Script de creación de la base de datos y las tablas . . . 18

Datos de ejemplo . . . 20

Consultas de verificación . . . 21

1. Introducción

Las heladerías son negocios de alta rotación: cada día se atienden muchos pedidos pequeños, se vende una gran variedad de productos (conos, copas, malteadas, paletas, toppings), se consume inventario perecedero y se coordina el trabajo de varios empleados. Cuando toda esa información se maneja en cuadernos, hojas sueltas o archivos aislados, resulta difícil saber qué se vendió, qué falta en el inventario o cuánto se compró a cada proveedor.

Este documento presenta el diseño de una base de datos relacional para un local de helados que desea centralizar la información de sus empleados, clientes, productos, proveedores, pedidos y facturas. El trabajo recorre las etapas del diseño de bases de datos: planteamiento del problema, objetivos, diccionario de datos, modelo entidad-relación (MER), modelo relacional (MR), normalización hasta la tercera forma normal (3FN) e implementación física en MySQL Workbench.

Alcance del sistema

El diseño incluye:

- Registro de empleados y de su cargo.
- Registro de clientes (un pedido puede hacerse sin cliente registrado).
- Catálogo de productos organizado por categorías, con precio y control de stock.
- Registro de proveedores y de las compras de mercancía que se les hacen.
- Registro de pedidos con el detalle de productos solicitados.
- Facturación de los pedidos, con el método de pago utilizado.

El diseño no incluye la interfaz gráfica, la nómina, la contabilidad general ni la integración con sistemas externos de facturación electrónica; esos elementos podrían construirse más adelante sobre esta base.

Propósito principal

Contar con una estructura de datos íntegra, sin redundancias innecesarias y lista para implementarse en MySQL, que permita registrar la operación diaria del local y consultarla con rapidez mediante SQL.

2. Planteamiento del problema

Situación actual

El local de helados lleva el control de su operación de forma manual o con herramientas dispersas: un cuaderno para los pedidos, hojas de cálculo para el inventario y facturas en papel. Esta forma de trabajar es suficiente cuando el volumen es bajo, pero a medida que el negocio crece empiezan a aparecer fallas.

Problemas identificados

- **Información duplicada o inconsistente:** el mismo cliente o proveedor se anota varias veces y con datos distintos (teléfonos o nombres diferentes).
- **Inventario poco confiable:** no existe un vínculo directo entre lo que se compra, lo que se vende y lo que realmente queda disponible.
- **Poca trazabilidad:** es difícil relacionar un pedido con el empleado que lo atendió, el cliente que lo pidió y la factura que se generó.
- **Consultas lentas o imposibles:** preguntas como «¿cuál fue el producto más vendido del mes?» o «¿cuánto se le compró a cada proveedor?» obligan a revisar los registros uno por uno.
- **Riesgo de pérdida de información:** papeles extraviados, archivos sobrescritos o datos que solo conoce una persona.

Justificación de la solución propuesta

Una base de datos relacional resuelve estos problemas porque centraliza la información en un solo lugar, garantiza la integridad mediante llaves primarias y foráneas, evita la duplicación gracias a la normalización, permite obtener reportes con consultas SQL y sirve como fundamento para una futura aplicación de gestión del local. Además, MySQL es un motor gratuito, ampliamente documentado y con una herramienta gráfica (MySQL Workbench) que facilita el modelado y la implementación.

Pregunta guía

¿Cómo diseñar una base de datos relacional, normalizada y de fácil consulta, que integre la información de empleados, clientes, productos, proveedores, pedidos y facturas de una heladería?

3. Macro con todos los módulos

El sistema se organiza en seis módulos. Cada uno agrupa las tablas que atienden una función del negocio y se conecta con los demás a través de llaves foráneas.

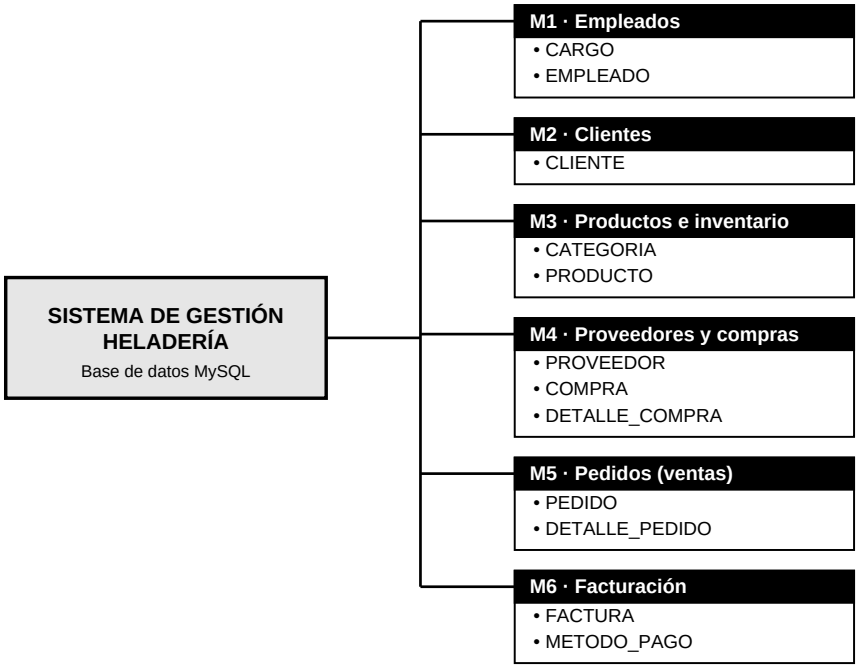


Figura 1. Mapa modular del sistema de gestión de la heladería.

Módulo	Tablas	Función principal
M1 · Empleados	CARGO, EMPLEADO	Administra el personal del local y el cargo que ocupa cada empleado.
M2 · Clientes	CLIENTE	Guarda los datos de los clientes registrados.
M3 · Productos e inventario	CATEGORIA, PRODUCTO	Catálogo de productos con precio de venta, estado y stock.
M4 · Proveedores y compras	PROVEEDOR, COMPRA, DETALLE_COMPRA	Registra los proveedores y la mercancía que se les compra.
M5 · Pedidos (ventas)	PEDIDO, DETALLE_PEDIDO	Registra cada pedido y los productos que incluye.
M6 · Facturación	FACTURA, METODO_PAGO	Genera la factura del pedido y registra la forma de pago.

Interacción entre módulos

- M1 → M5: los empleados atienden los pedidos.
- M2 → M5: los clientes realizan los pedidos.
- M3 → M5: los productos del catálogo se incluyen en los pedidos.
- M5 → M6: cada pedido facturado genera una factura.
- M4 → M3: las compras a proveedores abastecen los productos.
- M1 → M4: los empleados registran las compras.

> La actualización automática del stock al comprar o vender (por ejemplo, con triggers) queda como una mejora futura y no forma parte de este diseño.

4. Objetivos

Objetivo general

Diseñar una base de datos relacional en MySQL que permita gestionar de forma integrada los empleados, clientes, productos, proveedores, pedidos y facturas de una heladería.

Objetivos específicos

1. Identificar los requerimientos de información del local y las entidades que intervienen en su operación.
2. Definir el diccionario de datos con los atributos, tipos de dato, llaves y restricciones de cada entidad.
3. Modelar las relaciones entre las entidades mediante el modelo entidad-relación (MER) con sus cardinalidades.
4. Transformar el MER en un modelo relacional inicial.
5. Normalizar el modelo hasta la tercera forma normal (3FN) para eliminar redundancias y anomalías.
6. Implementar el modelo corregido en MySQL Workbench mediante un script SQL.

5. Diccionario de datos

El diccionario describe cada tabla del modelo final: sus campos, tipo de dato, llaves, restricciones y significado. Los tipos corresponden a MySQL 8.

Convenciones y supuestos

- Nombres en minúscula con guion bajo. La llave primaria se llama *id_<entidad>* y la llave foránea conserva el nombre de la llave que referencia.
- PK = llave primaria; FK = llave foránea. En DETALLE_PEDIDO y DETALLE_COMPRA la llave primaria es compuesta.
- Un pedido puede no tener cliente registrado (cliente ocasional), por eso PEDIDO.id_cliente admite NULL.
- DETALLE_PEDIDO.precio_unitario guarda el precio cobrado en el momento de la venta, porque el precio de PRODUCTO puede cambiar después.
- Los productos que se compran a proveedores y los que se venden se manejan en un mismo catálogo.
- Los valores monetarios se guardan como DECIMAL (nunca FLOAT) para evitar errores de redondeo.

Tabla: CARGO

Catálogo de cargos del personal y su salario base.

Campo	Tipo de dato	Llave	Nulo	Restricciones	Descripción
id_cargo	INT	PK	NOT NULL	AUTO_INCREMENT	Identificador único del cargo
nombre_cargo	VARCHAR(50)	—	NOT NULL	UNIQUE	Nombre del cargo (Administrador, Heladero, Cajero)
salario_base	DECIMAL(10,2)	—	NOT NULL	Mayor o igual a 0	Salario base mensual asociado al cargo
descripcion	VARCHAR(150)	—	NULL	—	Funciones generales del cargo

Tabla: EMPLEADO

Personas que trabajan en el local y atienden pedidos o registran compras.

Campo	Tipo de dato	Llave	Nulo	Restricciones	Descripción
id_empleado	INT	PK	NOT NULL	AUTO_INCREMENT	Identificador único del empleado
cedula	VARCHAR(15)	—	NOT NULL	UNIQUE	Documento de identidad del empleado
nombres	VARCHAR(60)	—	NOT NULL	—	Nombres del empleado
apellidos	VARCHAR(60)	—	NOT NULL	—	Apellidos del empleado
telefono	VARCHAR(15)	—	NOT NULL	—	Teléfono de contacto
correo	VARCHAR(100)	—	NULL	—	Correo electrónico
direccion	VARCHAR(120)	—	NULL	—	Dirección de residencia
fecha_contratacion	DATE	—	NOT NULL	—	Fecha de ingreso al local
estado	ENUM	—	NOT NULL	ACTIVO / INACTIVO. Por defecto ACTIVO	Estado laboral del empleado

Campo	Tipo de dato	Llave	Nulo	Restricciones	Descripción
id_cargo	INT	FK	NOT NULL	Referencia a CARGO	Cargo que desempeña el empleado

Tabla: CLIENTE

Clientes registrados del local (los clientes ocasionales no necesitan registro).

Campo	Tipo de dato	Llave	Nulo	Restricciones	Descripción
id_cliente	INT	PK	NOT NULL	AUTO_INCREMENT	Identificador único del cliente
documento	VARCHAR(15)	—	NOT NULL	UNIQUE	Documento de identidad o NIT del cliente
nombres	VARCHAR(60)	—	NOT NULL	—	Nombres del cliente
apellidos	VARCHAR(60)	—	NOT NULL	—	Apellidos del cliente
telefono	VARCHAR(15)	—	NULL	—	Teléfono de contacto
correo	VARCHAR(100)	—	NULL	—	Correo para envío de factura
direccion	VARCHAR(120)	—	NULL	—	Dirección (útil para domicilios)
fecha_registro	TIMESTAMP	—	NOT NULL	Por defecto CURRENT_TIMESTAMP	Fecha y hora en que se registró el cliente

Tabla: CATEGORIA

Catálogo de categorías de productos.

Campo	Tipo de dato	Llave	Nulo	Restricciones	Descripción
id_categoria	INT	PK	NOT NULL	AUTO_INCREMENT	Identificador único de la categoría
nombre	VARCHAR(50)	—	NOT NULL	UNIQUE	Nombre (Helados en cono, Copas, Malteadas...)
descripcion	VARCHAR(150)	—	NULL	—	Descripción de la categoría

Tabla: PRODUCTO

Artículos que el local vende y compra a sus proveedores.

Campo	Tipo de dato	Llave	Nulo	Restricciones	Descripción
id_producto	INT	PK	NOT NULL	AUTO_INCREMENT	Identificador único del producto
nombre	VARCHAR(80)	—	NOT NULL	—	Nombre comercial del producto
descripcion	VARCHAR(200)	—	NULL	—	Descripción, sabores o ingredientes
precio_venta	DECIMAL(10,2)	—	NOT NULL	Mayor o igual a 0	Precio de venta vigente al público
stock	INT	—	NOT NULL	Mayor o igual a 0. Por defecto 0	Unidades disponibles en inventario
estado	ENUM	—	NOT NULL	ACTIVO / INACTIVO. Por defecto ACTIVO	Indica si el producto se ofrece actualmente
id_categoria	INT	FK	NOT NULL	Referencia a CATEGORIA	Categoría a la que pertenece

Tabla: PROVEEDOR

Empresas o personas que abastecen al local.

Campo	Tipo de dato	Llave	Nulo	Restricciones	Descripción
id_proveedor	INT	PK	NOT NULL	AUTO_INCREMENT	Identificador único del proveedor
nit	VARCHAR(20)	—	NOT NULL	UNIQUE	NIT o documento tributario
razon_social	VARCHAR(100)	—	NOT NULL	—	Nombre o razón social
contacto	VARCHAR(80)	—	NULL	—	Persona de contacto
telefono	VARCHAR(15)	—	NOT NULL	—	Teléfono del proveedor
correo	VARCHAR(100)	—	NULL	—	Correo electrónico
direccion	VARCHAR(120)	—	NULL	—	Dirección del proveedor

Tabla: METODO_PAGO

Catálogo de formas de pago aceptadas.

Campo	Tipo de dato	Llave	Nulo	Restricciones	Descripción
id_metodo_pago	INT	PK	NOT NULL	AUTO_INCREMENT	Identificador único del método de pago
nombre	VARCHAR(30)	—	NOT NULL	UNIQUE	Efectivo, tarjeta, transferencia...

Tabla: PEDIDO

Solicitud de un cliente atendida por un empleado.

Campo	Tipo de dato	Llave	Nulo	Restricciones	Descripción
id_pedido	INT	PK	NOT NULL	AUTO_INCREMENT	Identificador único del pedido
fecha_hora	DATETIME	—	NOT NULL	Por defecto CURRENT_TIMESTAMP	Momento en que se registró el pedido
tipo_pedido	ENUM	—	NOT NULL	LOCAL / LLEVAR / DOMICILIO. Por defecto LOCAL	Modalidad de entrega
estado	ENUM	—	NOT NULL	PENDIENTE / PREPARADO / ENTREGADO / CANCELADO. Por defecto PENDIENTE	Estado actual del pedido
observaciones	VARCHAR(200)	—	NULL	—	Notas (sabores, sin topping, etc.)
id_cliente	INT	FK	NULL	Referencia a CLIENTE	Cliente que hizo el pedido (NULL si es ocasional)
id_empleado	INT	FK	NOT NULL	Referencia a EMPLEADO	Empleado que atendió el pedido

Tabla: DETALLE_PEDIDO

Productos incluidos en cada pedido (resuelve la relación M:N).

Campo	Tipo de dato	Llave	Nulo	Restricciones	Descripción
id_pedido	INT	PK, FK	NOT NULL	Referencia a PEDIDO	Pedido al que pertenece la línea
id_producto	INT	PK, FK	NOT NULL	Referencia a PRODUCTO	Producto solicitado
cantidad	INT	—	NOT NULL	Mayor que 0	Unidades solicitadas

Campo	Tipo de dato	Llave	Nulo	Restricciones	Descripción
precio_unitario	DECIMAL(10,2)	—	NOT NULL	Mayor o igual a 0	Precio cobrado en el momento de la venta

Tabla: FACTURA

Documento de cobro generado a partir de un pedido.

Campo	Tipo de dato	Llave	Nulo	Restricciones	Descripción
id_factura	INT	PK	NOT NULL	AUTO_INCREMENT	Identificador único interno
numero_factura	VARCHAR(20)	—	NOT NULL	UNIQUE	Número visible de la factura
fecha_emision	DATETIME	—	NOT NULL	Por defecto CURRENT_TIMESTAMP	Fecha y hora de emisión
subtotal	DECIMAL(12,2)	—	NOT NULL	Mayor o igual a 0	Suma de las líneas del pedido antes de IVA
valor_iva	DECIMAL(12,2)	—	NOT NULL	Mayor o igual a 0	Valor del impuesto liquidado
total	DECIMAL(12,2)	—	NOT NULL	Mayor o igual a 0	Valor total a pagar
id_pedido	INT	FK	NOT NULL	UNIQUE. Referencia a PEDIDO	Pedido facturado (relación 1:1)
id_metodo_pago	INT	FK	NOT NULL	Referencia a METODO_PAGO	Forma de pago utilizada

Tabla: COMPRA

Compra de mercancía a un proveedor.

Campo	Tipo de dato	Llave	Nulo	Restricciones	Descripción
id_compra	INT	PK	NOT NULL	AUTO_INCREMENT	Identificador único de la compra
fecha_compra	DATE	—	NOT NULL	—	Fecha en que se realizó la compra
numero_remision	VARCHAR(30)	—	NULL	—	Número de remisión o factura del proveedor
id_proveedor	INT	FK	NOT NULL	Referencia a PROVEEDOR	Proveedor al que se compró
id_empleado	INT	FK	NOT NULL	Referencia a EMPLEADO	Empleado que registró la compra

Tabla: DETALLE_COMPRA

Productos incluidos en cada compra (resuelve la relación M:N).

Campo	Tipo de dato	Llave	Nulo	Restricciones	Descripción
id_compra	INT	PK, FK	NOT NULL	Referencia a COMPRA	Compra a la que pertenece la línea
id_producto	INT	PK, FK	NOT NULL	Referencia a PRODUCTO	Producto comprado
cantidad	INT	—	NOT NULL	Mayor que 0	Unidades compradas
costo_unitario	DECIMAL(10,2)	—	NOT NULL	Mayor o igual a 0	Costo pagado por unidad

6. Modelo Entidad-Relación (MER)

A partir del problema se identificaron diez entidades: siete principales y tres catálogos (CARGO, CATEGORIA y METODO_PAGO). Los catálogos se modelan como entidades propias desde el MER, y no como un simple texto dentro de otra entidad, porque tienen identidad e información independiente (el cargo tiene su salario base, la categoría su descripción). Las entidades PEDIDO y COMPRA son el centro del modelo: una registra lo que se vende y la otra lo que se compra.

Entidad	Descripción	Identificador
EMPLEADO	Persona que trabaja en el local, atiende pedidos y registra compras.	id_empleado
CARGO	Puesto que ocupa un empleado, con su salario base (administrador, heladero, cajero).	id_cargo
CLIENTE	Persona que solicita productos al local.	id_cliente
CATEGORIA	Grupo al que pertenece un producto (helados en cono, copas, malteadas...).	id_categoria
PRODUCTO	Artículo que el local vende y compra.	id_producto
PEDIDO	Solicitud de productos atendida por un empleado.	id_pedido
FACTURA	Documento de cobro generado a partir de un pedido.	id_factura
METODO_PAGO	Forma de pago aceptada para cobrar una factura (efectivo, tarjeta...).	id_metodo_pago
PROVEEDOR	Empresa o persona que abastece al local.	id_proveedor
COMPRA	Adquisición de mercancía a un proveedor.	id_compra

Relaciones y cardinalidades

Relación	Entidades	Cardinalidad	Lectura
desempeña	CARGO – EMPLEADO	1 : N	Un cargo lo desempeñan muchos empleados; cada empleado tiene un solo cargo.
realiza	CLIENTE – PEDIDO	1 : N	Un cliente puede realizar muchos pedidos; un pedido pertenece a un solo cliente (o a ninguno si es ocasional).
atiende	EMPLEADO – PEDIDO	1 : N	Un empleado atiende muchos pedidos; cada pedido es atendido por un solo empleado.
clasifica	CATEGORIA – PRODUCTO	1 : N	Una categoría agrupa muchos productos; cada producto pertenece a una sola categoría.
contiene	PEDIDO – PRODUCTO	N : M	Un pedido contiene uno o varios productos y un producto aparece en muchos pedidos. Atributos propios: cantidad, precio_unitario.
genera	PEDIDO – FACTURA	1 : 1	Un pedido genera como máximo una factura (los pedidos cancelados no se facturan); cada factura corresponde a un único pedido.
paga	METODO_PAGO – FACTURA	1 : N	Un método de pago se usa en muchas facturas; cada factura se paga con un solo método.
provee	PROVEEDOR – COMPRA	1 : N	Un proveedor tiene muchas compras; cada compra se hace a un solo proveedor.
registra	EMPLEADO – COMPRA	1 : N	Un empleado registra muchas compras; cada compra la registra un solo empleado.
incluye	COMPRA – PRODUCTO	N : M	Una compra incluye uno o varios productos y un producto se compra muchas veces. Atributos propios: cantidad, costo_unitario.

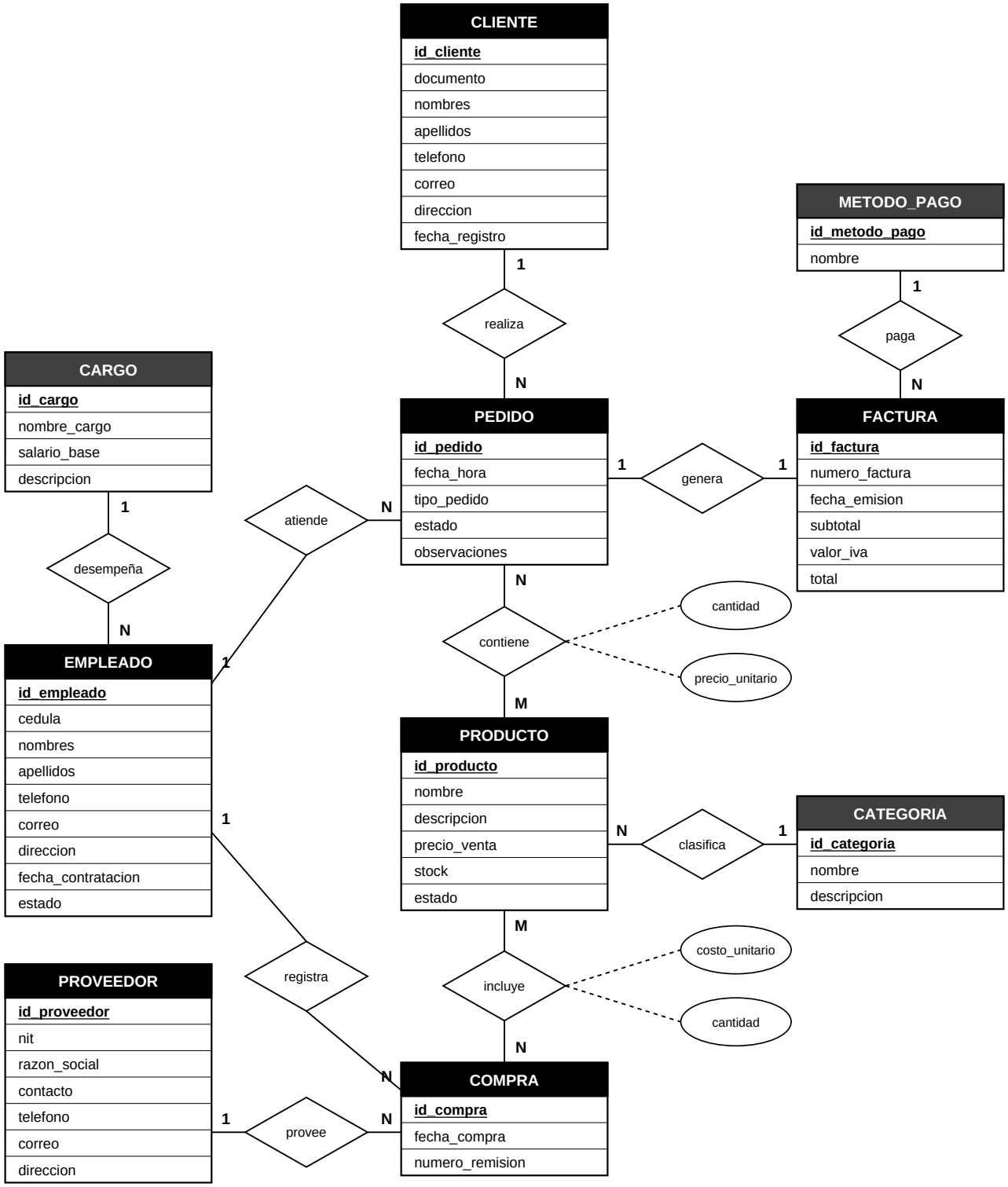


Figura 2. Modelo Entidad-Relación de la heladería. Los atributos se muestran dentro de cada entidad; el identificador está subrayado y los círculos son atributos de las relaciones N:M. Encabezado gris oscuro: catálogos (CARGO, CATEGORIA y METODO_PAGO).

7. Modelo Relacional inicial (MR)

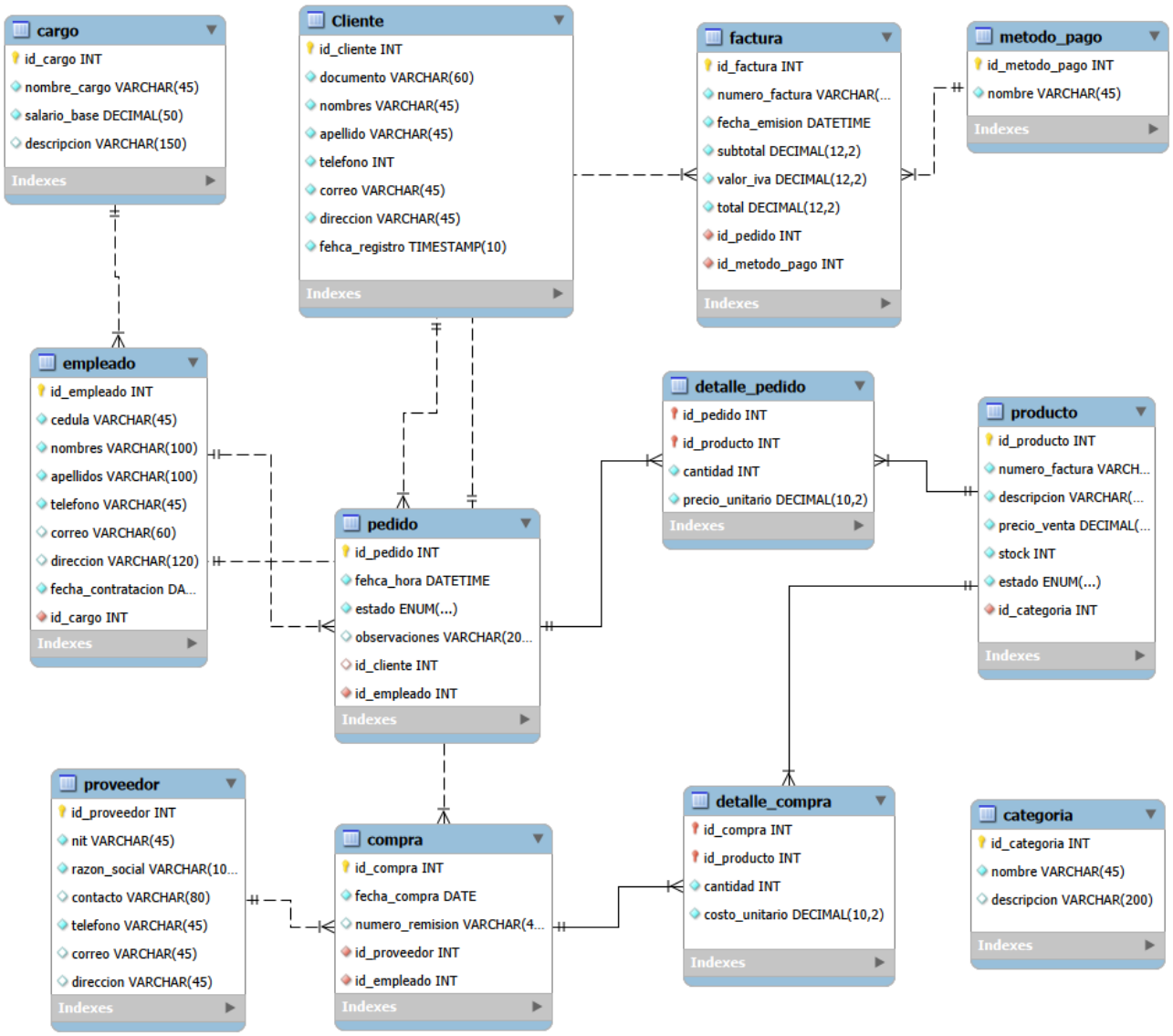


Figura 3. Diagrama MR elaborado con MySQL Workbench.

8. Proceso de normalización

La normalización organiza los datos para eliminar redundancias y evitar anomalías de inserción, actualización y borrado. Se aplican tres formas normales, cada una sobre el resultado de la anterior:

- **1FN:** todos los atributos contienen valores atómicos y no existen grupos repetitivos.
- **2FN:** está en 1FN y todo atributo que no es llave depende de toda la llave primaria (sin dependencias parciales).
- **3FN:** está en 2FN y ningún atributo que no es llave depende de otro atributo que tampoco lo es (sin dependencias transitivas).

Para mostrar el proceso se parte de una «ficha de pedido» tal como podría anotarse en papel, con todos los datos mezclados en un solo registro. Luego se revisa el MR inicial con los mismos criterios.

Las cifras y nombres de las tablas siguientes son ejemplos inventados con fines didácticos.

8.1 Punto de partida: ficha de pedido sin normalizar (UNF)

Un mismo registro contiene datos del pedido, del cliente, del empleado y una lista de productos. Los datos del cliente y del empleado se repiten en cada pedido, y la lista de productos es un grupo repetitivo.

id_ pedido	fecha_ hora	id_ cliente	nombre_ cliente	id_ empleado	nombre_ empleado	cargo	salario_ base	metodo_ pago	Productos (id_producto · nombre · categoria · cantidad · precio_unitario)
1001	2026-10-15	5	Laura Pérez	2	Carlos Ruiz	Heladero	1.500.000	Efectivo	P01 · Cono sencillo · Helados en cono · 2 · 4.500 P03 · Malteada de fresa · Malteadas · 1 · 9.000
1002	2026-10-15	5	Laura Pérez	2	Carlos Ruiz	Heladero	1.500.000	Tarjeta débito	P02 · Copa de la casa · Copas · 1 · 12.000

8.2 Primera forma normal (1FN)

Se elimina el grupo repetitivo: cada producto del pedido pasa a ocupar su propia fila y la llave primaria pasa a ser compuesta: (id_pedido, id_producto). Todos los valores son atómicos.

id_ pedido	id_ producto	fecha_ hora	id_ cliente	nombre_ cliente	id_ empleado	nombre_ empleado	cargo	salario_ base	metodo_ pago	nombre_ producto	categoria	cantidad	precio_ unitario
1001	P01	2026-10-15	5	Laura Pérez	2	Carlos Ruiz	Heladero	1.500.000	Efectivo	Cono sencillo	Helados en cono	2	4.500
1001	P03	2026-10-15	5	Laura Pérez	2	Carlos Ruiz	Heladero	1.500.000	Efectivo	Malteada de fresa	Malteadas	1	9.000
1002	P02	2026-10-15	5	Laura Pérez	2	Carlos Ruiz	Heladero	1.500.000	Tarjeta débito	Copa de la casa	Copas	1	12.000

Problema que persiste: el nombre del cliente, el empleado y su salario se repiten en cada fila. Si Laura cambia de nombre habría que corregirlo en muchos registros (anomalía de actualización).

8.3 Segunda forma normal (2FN)

Con llave compuesta (id_pedido, id_producto) se buscan atributos que dependan solo de una parte de la llave:

```
DF1: id_pedido → fecha_hora, id_cliente, nombre_cliente, id_empleado, nombre_empleado, cargo,
     salario_base, metodo_pago
DF2: id_producto → nombre_producto, categoria
DF3: (id_pedido, id_producto) → cantidad, precio_unitario
```

DF1 y DF2 son dependencias parciales (dependen de una sola parte de la llave), por eso se separan en tablas propias. Solo DF3 depende de la llave completa y permanece en el detalle.

Resultado en 2FN:

PEDIDO(id_pedido, fecha_hora, id_cliente, nombre_cliente, id_empleado, nombre_empleado, cargo, salario_base, metodo_pago)

PRODUCTO(id_producto, nombre_producto, categoria)

DETALLE_PEDIDO(#id_pedido, #id_producto, cantidad, precio_unitario)

8.4 Tercera forma normal (3FN)

En PEDIDO y PRODUCTO todavía hay atributos que no dependen directamente de la llave primaria, sino de otro atributo (dependencias transitivas):

```
id_pedido → id_cliente → nombre_cliente
id_pedido → id_empleado → nombre_empleado, cargo
id_empleado → cargo → salario_base   (el salario depende del cargo, no de la persona)
id_producto → categoria              (se extrae a un catálogo para evitar textos inconsistentes)
id_pedido → metodo_pago              (pertenece al cobro; pasa a FACTURA con catálogo propio)
```

Cada grupo de atributos se mueve a su propia tabla y se deja solo la llave foránea. Resultado en 3FN:

CARGO(id_cargo, nombre_cargo, salario_base, descripcion)

EMPLEADO(id_empleado, cedula, nombres, apellidos, ..., #id_cargo)

CLIENTE(id_cliente, documento, nombres, apellidos, ...)

CATEGORIA(id_categoria, nombre, descripcion)

PRODUCTO(id_producto, nombre, ..., precio_venta, stock, #id_categoria)

METODO_PAGO(id_metodo_pago, nombre)

PEDIDO(id_pedido, fecha_hora, ..., #id_cliente, #id_empleado)

DETALLE_PEDIDO(#id_pedido, #id_producto, cantidad, precio_unitario)

FACTURA(id_factura, numero_factura, ..., #id_pedido, #id_metodo_pago)

> CARGO, CATEGORIA y METODO_PAGO ya aparecen como entidades en el MER (sección 6). Aquí se muestra cómo surgirían si el diseño partiera de la ficha en papel; como el modelo nació del MER, el MR inicial ya las incluye y no hay que «descubrirlas» después.

8.5 Aplicación al MR inicial

Al revisar las doce tablas del MR inicial con los criterios anteriores se encontraron estos hallazgos:

#	Tabla.atributo	Problema detectado	Forma normal	Corrección aplicada
1	PEDIDO. nombre_cliente	id_pedido → id_cliente → nombre_cliente (dependencia transitiva).	3FN	Se elimina. El nombre se obtiene con un JOIN a CLIENTE.
2	DETALLE_PEDIDO. nombre_producto	Depende solo de id_producto, que es parte de la llave compuesta.	2FN	Se elimina; ya existe en PRODUCTO.
3	DETALLE_PEDIDO. subtotal	Atributo derivado: cantidad × precio_unitario.	3FN (redundancia)	Se elimina; se calcula en las consultas.
4	DETALLE_COMPRA. nombre_producto	Igual que el hallazgo 2.	2FN	Se elimina; ya existe en PRODUCTO.
5	CARGO, CATEGORIA, METODO_PAGO, EMPLEADO, PRODUCTO y FACTURA	El salario depende del cargo, y la categoría y el método de pago son catálogos: ya se identificaron como entidades en el MER.	—	Sin cambios: EMPLEADO, PRODUCTO y FACTURA solo guardan la llave foránea; ya cumplían 1FN, 2FN y 3FN.

#	Tabla.atributo	Problema detectado	Forma normal	Corrección aplicada
6	CLIENTE, PROVEEDOR, COMPRA y resto de PEDIDO	Todos sus atributos dependen únicamente de la llave primaria.	—	Sin cambios: ya cumplían 1FN, 2FN y 3FN.

Desnormalización controlada: FACTURA.subtotal, valor_iva y total se conservan aunque se podrían calcular, porque la factura es un documento histórico que no debe cambiar si más adelante se modifican los precios o la tarifa del impuesto. Por la misma razón, DETALLE_PEDIDO.precio_unitario guarda el precio de la venta y no depende del precio actual de PRODUCTO.

8.6 Verificación final del modelo

Tabla	1FN	2FN	3FN	Justificación
CARGO	Cumple	Cumple	Cumple	Todo depende de id_cargo.
EMPLEADO	Cumple	Cumple	Cumple	El salario ya no está aquí; solo se guarda id_cargo.
CLIENTE	Cumple	Cumple	Cumple	Todo depende de id_cliente.
CATEGORIA	Cumple	Cumple	Cumple	Todo depende de id_categoria.
PRODUCTO	Cumple	Cumple	Cumple	La categoría se referencia por id_categoria.
PROVEEDOR	Cumple	Cumple	Cumple	Todo depende de id_proveedor.
METODO_PAGO	Cumple	Cumple	Cumple	Catálogo de dos campos.
PEDIDO	Cumple	Cumple	Cumple	Cliente y empleado se referencian por llave foránea.
DETALLE_PEDIDO	Cumple	Cumple	Cumple	cantidad y precio_unitario dependen de (id_pedido, id_producto).
FACTURA	Cumple	Cumple	Cumple	Atributos de cobro; el pago se referencia por llave foránea.
COMPRA	Cumple	Cumple	Cumple	Proveedor y empleado se referencian por llave foránea.
DETALLE_COMPRA	Cumple	Cumple	Cumple	cantidad y costo_unitario dependen de (id_compra, id_producto).

9. Modelo Relacional corregido (tras la normalización)

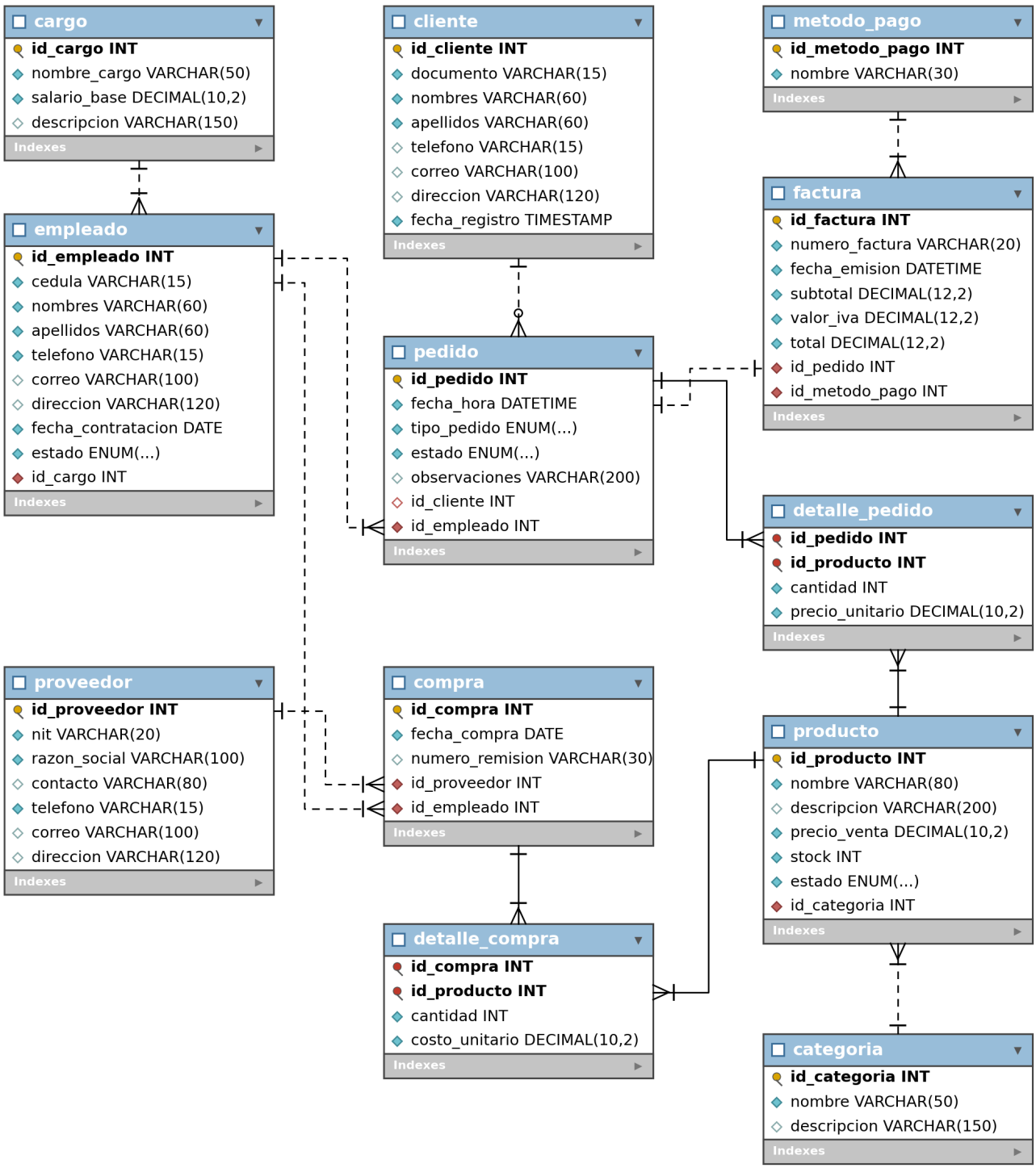


Figura 4. Modelo relacional corregido (3FN).

10. Implementación en MySQL Workbench

El modelo corregido se implementa en MySQL 8.x con el siguiente script. Para ejecutarlo, se abre una pestaña de consultas en MySQL Workbench (File → New Query Tab), se pega el script completo y se ejecuta con el botón del rayo.

Script de creación de la base de datos y las tablas

Las tablas se crean en orden: primero las que no dependen de otras (catálogos) y al final las que tienen llaves foráneas.

0. Base de datos

```
-- ============================================================
-- Proyecto: Base de datos para una heladería
-- Motor: MySQL 8.x          Herramienta: MySQL Workbench
-- ============================================================
CREATE DATABASE IF NOT EXISTS heladeria_db
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE heladeria_db;
```

1. Módulo de empleados

```
CREATE TABLE cargo (
    id_cargo      INT           NOT NULL AUTO_INCREMENT,
    nombre_cargo  VARCHAR(50)   NOT NULL,
    salario_base  DECIMAL(10,2) NOT NULL,
    descripcion   VARCHAR(150)  NULL,
    PRIMARY KEY (id_cargo),
    UNIQUE KEY uq_cargo_nombre (nombre_cargo),
    CONSTRAINT ck_cargo_salario CHECK (salario_base >= 0)
) ENGINE=InnoDB;

CREATE TABLE empleado (
    id_empleado        INT          NOT NULL AUTO_INCREMENT,
    cedula             VARCHAR(15)  NOT NULL,
    nombres            VARCHAR(60)  NOT NULL,
    apellidos          VARCHAR(60)  NOT NULL,
    telefono           VARCHAR(15)  NOT NULL,
    correo             VARCHAR(100) NULL,
    direccion          VARCHAR(120) NULL,
    fecha_contratacion DATE         NOT NULL,
    estado             ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',
    id_cargo           INT          NOT NULL,
    PRIMARY KEY (id_empleado),
    UNIQUE KEY uq_empleado_cedula (cedula),
    CONSTRAINT fk_empleado_cargo FOREIGN KEY (id_cargo)
        REFERENCES cargo (id_cargo) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;
```

2. Módulo de clientes

```
CREATE TABLE cliente (
    id_cliente     INT          NOT NULL AUTO_INCREMENT,
    documento      VARCHAR(15)  NOT NULL,
    nombres        VARCHAR(60)  NOT NULL,
    apellidos      VARCHAR(60)  NOT NULL,
    telefono       VARCHAR(15)  NULL,
    correo         VARCHAR(100) NULL,
    direccion      VARCHAR(120) NULL,
    fecha_registro TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id_cliente),
    UNIQUE KEY uq_cliente_documento (documento)
) ENGINE=InnoDB;
```

3. Módulo de productos e inventario

```
CREATE TABLE categoria (
    id_categoria INT          NOT NULL AUTO_INCREMENT,
    nombre       VARCHAR(50)  NOT NULL,
    descripcion  VARCHAR(150) NULL,
    PRIMARY KEY (id_categoria),
    UNIQUE KEY uq_categoria_nombre (nombre)
) ENGINE=InnoDB;

CREATE TABLE producto (
    id_producto  INT           NOT NULL AUTO_INCREMENT,
    nombre       VARCHAR(80)   NOT NULL,
    descripcion  VARCHAR(200)  NULL,
    precio_venta DECIMAL(10,2) NOT NULL,
    stock        INT           NOT NULL DEFAULT 0,
    estado       ENUM('ACTIVO','INACTIVO') NOT NULL DEFAULT 'ACTIVO',
    id_categoria INT           NOT NULL,
    PRIMARY KEY (id_producto),
    CONSTRAINT ck_producto_precio CHECK (precio_venta >= 0),
    CONSTRAINT ck_producto_stock  CHECK (stock >= 0),
    CONSTRAINT fk_producto_categoria FOREIGN KEY (id_categoria)
        REFERENCES categoria (id_categoria) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;
```

4. Módulo de proveedores y compras

```
CREATE TABLE proveedor (
    id_proveedor INT          NOT NULL AUTO_INCREMENT,
    nit          VARCHAR(20)  NOT NULL,
    razon_social VARCHAR(100) NOT NULL,
    contacto     VARCHAR(80)  NULL,
    telefono     VARCHAR(15)  NOT NULL,
    correo       VARCHAR(100) NULL,
    direccion    VARCHAR(120) NULL,
    PRIMARY KEY (id_proveedor),
    UNIQUE KEY uq_proveedor_nit (nit)
) ENGINE=InnoDB;

CREATE TABLE compra (
    id_compra       INT         NOT NULL AUTO_INCREMENT,
    fecha_compra    DATE        NOT NULL,
    numero_remision VARCHAR(30) NULL,
    id_proveedor    INT         NOT NULL,
    id_empleado     INT         NOT NULL,
    PRIMARY KEY (id_compra),
    CONSTRAINT fk_compra_proveedor FOREIGN KEY (id_proveedor)
        REFERENCES proveedor (id_proveedor) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_compra_empleado FOREIGN KEY (id_empleado)
        REFERENCES empleado (id_empleado) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE detalle_compra (
    id_compra      INT           NOT NULL,
    id_producto    INT           NOT NULL,
    cantidad       INT           NOT NULL,
    costo_unitario DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (id_compra, id_producto),
    CONSTRAINT ck_dcompra_cantidad CHECK (cantidad > 0),
    CONSTRAINT ck_dcompra_costo    CHECK (costo_unitario >= 0),
    CONSTRAINT fk_dcompra_compra FOREIGN KEY (id_compra)
        REFERENCES compra (id_compra) ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_dcompra_producto FOREIGN KEY (id_producto)
        REFERENCES producto (id_producto) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;
```

5. Módulo de pedidos (ventas)

```
CREATE TABLE pedido (
    id_pedido     INT      NOT NULL AUTO_INCREMENT,
    fecha_hora    DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    tipo_pedido   ENUM('LOCAL','LLEVAR','DOMICILIO') NOT NULL DEFAULT 'LOCAL',
    estado        ENUM('PENDIENTE','PREPARADO','ENTREGADO','CANCELADO')
                  NOT NULL DEFAULT 'PENDIENTE',
    observaciones VARCHAR(200) NULL,
    id_cliente    INT      NULL,          -- NULL = cliente ocasional
    id_empleado   INT      NOT NULL,
    PRIMARY KEY (id_pedido),
    CONSTRAINT fk_pedido_cliente FOREIGN KEY (id_cliente)
        REFERENCES cliente (id_cliente) ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_pedido_empleado FOREIGN KEY (id_empleado)
        REFERENCES empleado (id_empleado) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;

CREATE TABLE detalle_pedido (
    id_pedido       INT           NOT NULL,
    id_producto     INT           NOT NULL,
    cantidad        INT           NOT NULL,
    precio_unitario DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (id_pedido, id_producto),
    CONSTRAINT ck_dpedido_cantidad CHECK (cantidad > 0),
    CONSTRAINT ck_dpedido_precio   CHECK (precio_unitario >= 0),
    CONSTRAINT fk_dpedido_pedido FOREIGN KEY (id_pedido)
        REFERENCES pedido (id_pedido) ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_dpedido_producto FOREIGN KEY (id_producto)
        REFERENCES producto (id_producto) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;
```

6. Módulo de facturación

```
CREATE TABLE metodo_pago (
    id_metodo_pago INT         NOT NULL AUTO_INCREMENT,
    nombre         VARCHAR(30) NOT NULL,
    PRIMARY KEY (id_metodo_pago),
    UNIQUE KEY uq_metodo_pago_nombre (nombre)
) ENGINE=InnoDB;

CREATE TABLE factura (
    id_factura     INT           NOT NULL AUTO_INCREMENT,
    numero_factura VARCHAR(20)   NOT NULL,
    fecha_emision  DATETIME      NOT NULL DEFAULT CURRENT_TIMESTAMP,
    subtotal       DECIMAL(12,2) NOT NULL,
    valor_iva      DECIMAL(12,2) NOT NULL,
    total          DECIMAL(12,2) NOT NULL,
    id_pedido      INT           NOT NULL,
    id_metodo_pago INT           NOT NULL,
    PRIMARY KEY (id_factura),
    UNIQUE KEY uq_factura_numero (numero_factura),
    UNIQUE KEY uq_factura_pedido (id_pedido),      -- relación 1:1
    CONSTRAINT ck_factura_valores CHECK (subtotal >= 0 AND valor_iva >= 0 AND total >= 0),
    CONSTRAINT fk_factura_pedido FOREIGN KEY (id_pedido)
        REFERENCES pedido (id_pedido) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_factura_metodo FOREIGN KEY (id_metodo_pago)
        REFERENCES metodo_pago (id_metodo_pago) ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB;
```

Datos de ejemplo

Datos mínimos para probar el modelo. El porcentaje de IVA (19 %) es solo un ejemplo; debe ajustarse a la tarifa que corresponda al producto y al tipo de negocio.

```
-- ---------- Catálogos ----------
INSERT INTO cargo (nombre_cargo, salario_base, descripcion) VALUES
    ('Administrador', 2500000.00, 'Gestión general del local'),
    ('Heladero',      1500000.00, 'Prepara y despacha helados'),
    ('Cajero',        1500000.00, 'Atiende pedidos y factura');
```

```
INSERT INTO categoria (nombre, descripcion) VALUES
    ('Helados en cono', 'Conos sencillos y dobles'),
    ('Copas',           'Copas y sundaes'),
    ('Malteadas',       'Bebidas heladas');

INSERT INTO metodo_pago (nombre) VALUES
    ('Efectivo'), ('Tarjeta débito'), ('Tarjeta crédito'), ('Transferencia');

-- ---------- Datos de ejemplo ----------
INSERT INTO empleado (cedula, nombres, apellidos, telefono, fecha_contratacion, id_cargo)
VALUES ('1000000001', 'Carlos', 'Ruiz', '3001112233', '2026-01-15', 2);

INSERT INTO cliente (documento, nombres, apellidos, telefono)
VALUES ('1000000002', 'Laura', 'Pérez', '3109876543');

INSERT INTO producto (nombre, descripcion, precio_venta, stock, id_categoria) VALUES
    ('Cono sencillo',     'Una bola a elección',  4500.00, 100, 1),
    ('Copa de la casa',   'Tres bolas y topping', 12000.00, 40, 2),
    ('Malteada de fresa', 'Vaso de 12 oz',         9000.00, 30, 3);

INSERT INTO proveedor (nit, razon_social, contacto, telefono)
VALUES ('900000001-1', 'Lácteos del Norte S.A.S.', 'Marta Gómez', '3205550101');

INSERT INTO compra (fecha_compra, numero_remision, id_proveedor, id_empleado)
VALUES (CURDATE(), 'REM-0001', 1, 1);

INSERT INTO detalle_compra (id_compra, id_producto, cantidad, costo_unitario)
VALUES (1, 1, 50, 1800.00);

-- Pedido de ejemplo: 2 conos + 1 malteada = 18.000 (IVA de ejemplo al 19%)
INSERT INTO pedido (tipo_pedido, id_cliente, id_empleado) VALUES ('LOCAL', 1, 1);

INSERT INTO detalle_pedido (id_pedido, id_producto, cantidad, precio_unitario) VALUES
    (1, 1, 2, 4500.00),
    (1, 3, 1, 9000.00);

INSERT INTO factura (numero_factura, subtotal, valor_iva, total, id_pedido, id_metodo_pago)
VALUES ('F-0001', 18000.00, 3420.00, 21420.00, 1, 1);
```

Consultas de verificación

Ventas totales por producto

```
SELECT p.nombre,
       SUM(dp.cantidad)                    AS unidades_vendidas,
       SUM(dp.cantidad * dp.precio_unitario) AS total_vendido
FROM detalle_pedido dp
JOIN producto p ON p.id_producto = dp.id_producto
GROUP BY p.id_producto, p.nombre
ORDER BY total_vendido DESC;
```

Facturas con cliente, empleado y método de pago

```
SELECT f.numero_factura, f.fecha_emision,
       COALESCE(CONCAT(c.nombres, ' ', c.apellidos), 'Cliente ocasional') AS cliente,
       CONCAT(e.nombres, ' ', e.apellidos) AS atendido_por,
       mp.nombre AS metodo_pago, f.total
FROM factura f
JOIN pedido pe       ON pe.id_pedido = f.id_pedido
LEFT JOIN cliente c  ON c.id_cliente = pe.id_cliente
JOIN empleado e      ON e.id_empleado = pe.id_empleado
JOIN metodo_pago mp  ON mp.id_metodo_pago = f.id_metodo_pago;
```

Productos con stock bajo (menos de 20 unidades)

```
SELECT p.nombre, c.nombre AS categoria, p.stock
FROM producto p
JOIN categoria c ON c.id_categoria = p.id_categoria
WHERE p.stock < 20 AND p.estado = 'ACTIVO'
ORDER BY p.stock;
```